

ODOO BACKEND DEVELOPMENT ESSENTIALS

# Odoo Core Development

Install, modules, fields, and ORM basics

---

Merit Advisory

Odoo Full-Stack Developer Program

Beginner-friendly lecture materials

July 2026

July 2026



# Agenda

- 1 Section 1: Odoo 17 Install Overview to Hide the Database List — 7 topics
- 2 Section 2: Understand the Master Password to Recognize Module Folder Structure — 7 topics
- 3 Section 3: Define a Model with `_name` to Use an Integer Field — 7 topics
- 4 Section 4: Use a Float Field to Create a Compute Field — 7 topics
- 5 Section 5: Create a Related Field to Count Records with `search_count` — 7 topics
- 6 Section 6: Use `browse()` and `exists()` to Use `sudo` with Caution — 7 topics

## 7 Section 7: Create Related Record with [0, 0, vals] to Introduce CSV Access Rights — 7 topics

# 1 Section 1

Odoo 17 Install Overview to Hide the Database List

# 1.1 Odoo 17 Install Overview

- Odoo needs Python, PostgreSQL, and system libraries.
- A source install is useful for training.
- Use a virtual environment for Python packages.

## Clone source

```
git clone --depth 1 --branch 17.0 https://github.com/odoo/
```

## Install requirements

```
python3 -m venv venv  
source venv/bin/activate  
pip install -r requirements.txt
```

## 1.2 Start Odoo with odoo-bin

- odoo-bin starts the Odoo server.
- Run it from the source checkout.
- Use explicit options while learning.

### Start with config

```
./odoo-bin -c /etc/odoo17.conf
```

### Start one database

```
./odoo-bin -d odoo17_training --addons-path=addons
```

## 1.3 Understand odoo.conf Basics

- The config file stores common startup options.
- It keeps long commands readable.
- Use one config per environment.

### Minimal config

```
[options]  
addons_path = /opt/odoo/odoo17/addons  
db_host = 127.0.0.1
```

### Run with config

```
./odoo-bin -c ./odoo17.conf
```

## 1.4 Set db\_host, db\_user, and db\_password

- These options tell Odoo how to reach PostgreSQL.
- Use a dedicated PostgreSQL role.
- Prefer explicit local host settings in training.

### Config file values

```
db_host = 127.0.0.1  
db_user = odoo17  
db_password = change-me
```

### CLI values

```
./odoo-bin --db_host=127.0.0.1 --db_user=odoo17 --db_password=change-me
```



## 1.5 Configure addons\_path

- addons\_path tells Odoo where modules live.
- Separate core and custom addons clearly.
- Path order matters when names collide.

### Config addons path

```
addons_path = /opt/odoo/odoo17/addons,/opt/odoo/custom-addons
```

### CLI addons path

```
./odoo-bin --addons-path=addons,../custom-addons -d training
```

## 1.6 Write Logs to a File

- Logs show startup and runtime problems.
- A logfile is easier to inspect later.
- Keep permissions simple for the Odoo user.

### Config logfile

```
logfile = /var/log/odoo/odoo17.log
```

### View recent logs

```
less /var/log/odoo/odoo17.log
```

## 1.7 Hide the Database List

- list\_db controls the web database selector.
- Set it false on shared or public servers.
- It does not replace real access control.

### Disable listing

```
list_db = False
```

### Start with one database

```
./odoo-bin -c odoo17.conf -d odoo17_prod
```

# 2 Section 2

Understand the Master Password to Recognize Module Folder Structure

## 2.1 Understand the Master Password

- The master password protects database manager actions. [Config setting](#)
- Use a strong value outside source control.
- Do not confuse it with user passwords.

```
admin_passwd = replace-with-a-strong-secret
```

### Database manager use

```
Open /web/database/manager  
Enter the master password for backup or restore actions
```

## 2.2 Install a Module with -i

- i installs a module into a database.
- Use it when the module is not installed yet.
- Combine with stop-after-init for scripts.

### Install base CRM

```
./odoo-bin -d training -i crm --stop-after-init
```

### Install custom module

```
./odoo-bin -c odoo17.conf -d training -i estate --stop-aft
```

## 2.3 Upgrade a Module with -u

- -u updates an installed module.
- Use it after changing models, views, or data.
- Back up important databases first.

### Upgrade one module

```
./odoo-bin -d training -u estate --stop-after-init
```

### Upgrade several modules

```
./odoo-bin -c odoo17.conf -d training -u estate,sale --stop-after-init
```

## 2.4 Use shell -d

- The Odoo shell opens an ORM-aware Python prompt.
- Use -d to choose the database.
- It is useful for safe inspection and small fixes.

### Open shell

```
./odoo-bin shell -d training
```

### Inspect a model

```
env['res.partner'].search([], limit=3)
```



## 2.5 Scaffold a New Module

- Scaffold creates a starter module folder.
- It gives common files and directories.
- Review generated files before editing.

### Create module skeleton

```
./odoo-bin scaffold estate ../custom-addons
```

### Generated path

```
../custom-addons/estate/__manifest__.py  
../custom-addons/estate/models/models.py
```

## 2.6 Set Manifest name, version, and depends

- The manifest describes the module.
- depends controls module load order.
- Keep the version meaningful for releases.

### Basic manifest

```
{  
    'name': 'Estate',  
    'version': '17.0.1.0.0',  
    'depends': ['base'],  
}
```

### Depend on sales

```
{  
    'name': 'Estate Sale',  
    'depends': ['estate', 'sale'],  
}
```

## 2.7 Recognize Module Folder Structure

- Python models usually live in models.
- XML views usually live in views.
- Security files usually live in security.

### Common folders

```
estate/  
  __manifest__.py  
  models/  
  views/  
  security/
```

### Import models

```
# estate/models/__init__.py  
from . import estate_property
```

# 3 Section 3

Define a Model with `_name` to Use an Integer Field

## 3.1 Define a Model with `_name`

- `_name` gives the model its technical name.
- Use dotted names such as `estate.property`.
- The name is used by views, access rules, and code.

### New model

```
class EstateProperty(models.Model):  
    _name = 'estate.property'
```

### Use model name

```
self.env['estate.property'].search([])
```

## 3.2 Add a Model \_description

- \_description gives a readable model label.
- It helps logs and technical screens.
- Keep it short and human friendly.

### Description line

```
class EstateProperty(models.Model):  
    _name = 'estate.property'  
    _description = 'Estate Property'
```

### Another description

```
class EstateOffer(models.Model):  
    _name = 'estate.offer'  
    _description = 'Estate Offer'
```

## 3.3 Use a Char Field

- Char stores short text.
- Use it for names, codes, and labels.
- Add required=True when the value must exist.

### Name field

```
name = fields.Char(string='Name', required=True)
```

### Reference field

```
reference = fields.Char(string='Reference', copy=False)
```

## 3.4 Use a Text Field

- Text stores longer text.
- Use it for notes and descriptions.
- It is not limited like short labels.

### Description field

```
description = fields.Text(string='Description')
```

### Internal notes

```
note = fields.Text(string='Internal Notes')
```



## 3.5 Use a Boolean Field

- Boolean stores true or false.
- Use it for flags and toggles.
- Set a default when useful.

### Active flag

```
is_published = fields.Boolean(string='Published')
```

### Default true

```
active = fields.Boolean(default=True)
```

## 3.6 Use a Selection Field

- Selection stores one value from a fixed list.
- Each option has a key and label.
- Use it for simple states or categories.

### State field

```
state = fields.Selection([
    ('new', 'New'),
    ('done', 'Done'),
], default='new')
```

### Priority field

```
priority = fields.Selection([
    ('0', 'Normal'),
    ('1', 'High'),
])
```

## 3.7 Use an Integer Field

- Integer stores whole numbers.
- Use it for counts and quantities.
- It defaults to zero when empty.

### Quantity field

```
quantity = fields.Integer(string='Quantity')
```

### Bedrooms field

```
bedrooms = fields.Integer(default=2)
```

# 4 Section 4

Use a Float Field to Create a Compute Field

## 4.1 Use a Float Field

- Float stores decimal numbers.
- Use it for measurements and rates.
- Use monetary fields for accounting amounts.

### Area field

```
living_area = fields.Float(string='Living Area')
```

### Percentage field

```
commission_rate = fields.Float(string='Commission Rate')
```

## 4.2 Use a Date Field

- Date stores a calendar day.
- Use it when time of day is not needed.
- Odoo has helpers for today.

### Simple date

```
available_date = fields.Date(string='Available From')
```

### Default today

```
date_order = fields.Date(default=fields.Date.today)
```

## 4.3 Use a Datetime Field

- Datetime stores date and time.
- Use it for events and timestamps.
- Odoo handles display with user timezone.

### Meeting time

```
meeting_at = fields.Datetime(string='Meeting Time')
```

### Default now

```
created_marker = fields.Datetime(default=fields.Datetime.now)
```

## 4.4 Use a Many2one Field

- Many2one links this record to one other record.
- It creates a database foreign key.
- Use it for owner, partner, and category links.

### Partner link

```
partner_id = fields.Many2one('res.partner', string='Custom
```

### Required salesperson

```
user_id = fields.Many2one('res.users', required=True)
```



## 4.5 Use a One2many Field

- One2many shows records pointing back to this record.
- It needs the child model and inverse field.
- It is commonly used for line items.

### Order lines

```
line_ids = fields.One2many('sale.order.line', 'order_id')
```

### Estate offers

```
offer_ids = fields.One2many('estate.offer', 'property_id')
```

## 4.6 Use a Many2many Field

- Many2many links many records on both sides.
- Odoo stores links in a relation table.
- Use it for tags, followers, and allowed users.

### Tag field

```
tag_ids = fields.Many2many('estate.property.tag', string='')
```

### Allowed users

```
allowed_user_ids = fields.Many2many('res.users')
```

## 4.7 Create a Compute Field

- A compute field gets its value from a method.
- Use depends to declare trigger fields.
- Store only when search or performance needs it.

### Compute total

```
total_area = fields.Float(compute='_compute_total_area')

@api.depends('living_area', 'garden_area')
def _compute_total_area(self):
    for record in self:
        record.total_area = record.living_area + record.garden_area
```

### Stored compute

```
best_price = fields.Float(compute='_compute_best_price', store=True)
```

# 5 Section 5

Create a Related Field to Count Records with search\_count

## 5.1 Create a Related Field

- A related field follows a relationship path.
- It avoids duplicating simple values.
- Use store=True when filtering often.

### Partner email

```
partner_email = fields.Char(related='partner_id.email')
```

### Stored company

```
company_id = fields.Many2one(related='user_id.company_id',
```

## 5.2 Understand the create Override Idea

- create runs when new records are made.
- Call super to keep standard behavior.
- Use it for validation or default adjustments.

### Basic create override

```
@api.model_create_multi
def create(self, vals_list):
    records = super().create(vals_list)
    return records
```

### Fill a missing value

```
for vals in vals_list:
    vals.setdefault('state', 'new')
```

## 5.3 Understand the write Override Idea

- write runs when existing records change.
- Call super once for the normal update.
- Be careful with recursion and side effects.

### Basic write override

```
def write(self, vals):  
    result = super().write(vals)  
    return result
```

### React to field change

```
if 'state' in vals:  
    self.message_post(body='State changed')
```

## 5.4 Understand the unlink Idea

- unlink runs when records are deleted.
- Use it to block unsafe deletion.
- Archive records when deletion is not appropriate.

### Basic unlink override

```
def unlink(self):  
    return super().unlink()
```

### Block deletion

```
if any(record.state == 'done' for record in self):  
    raise UserError('Done records cannot be deleted.')
```



## 5.5 Find Records with search()

- search returns a recordset.
- Pass a domain to filter records.
- Use limit for small inspections.

### Find customers

```
customers = self.env['res.partner'].search([('customer_rank', > 0)])
```

### Find one record

```
partner = self.env['res.partner'].search([('email', '=', 'john.doe@example.com')], limit=1)
```

## 5.6 Build a domain = Filter

- A domain is a list of filter conditions.
- Each condition is field, operator, value.
- Domains are used by search, views, and rules.

### Simple domain

```
domain = [('state', '=', 'sale')]
```

### Multiple conditions

```
domain = [('amount_total', '>=', 1000), ('partner_id', '!=', ...)]
```

## 5.7 Count Records with search\_count

- search\_count returns an integer.
- It is faster than loading records just to count.
- Use the same domain style as search.

### Count customers

```
count = self.env['res.partner'].search_count([('customer_r
```

### Count open tasks

```
open_tasks = self.env['project.task'].search_count([('stag
```

# 6 Section 6

Use browse() and exists() to Use sudo with Caution

## 6.1 Use browse() and exists()

- browse creates a recordset from ids.
- It does not guarantee records exist.
- exists removes missing records from the set.

### Browse by id

```
partner = self.env['res.partner'].browse(42)
```

### Check existence

```
partner = self.env['res.partner'].browse(42).exists()
```

## 6.2 Use ensure\_one

- ensure\_one confirms the recordset has one record.
- It raises an error for zero or many records.
- Use it in methods designed for one record.

### Single-record method

```
def action_confirm(self):  
    self.ensure_one()  
    self.state = 'confirmed'
```

### Read one value

```
self.ensure_one()  
return self.name
```

## 6.3 Use mapped

- mapped extracts a field from each record.
- It can follow relational paths.
- It keeps recordset code compact.

### Names list

```
names = partners.mapped('name')
```

### Related records

```
companies = partners.mapped('company_id')
```

## 6.4 Use filtered

- filtered keeps records matching Python logic.
- Use search domains when the database can filter.
- Use filtered for already-loaded recordsets.

### Keep active records

```
active_partners = partners.filtered(lambda p: p.active)
```

### Keep high value orders

```
large_orders = orders.filtered(lambda o: o.amount_total >
```



## 6.5 Use active for Archive Behavior

- The active field controls archive behavior.
- Archived records are hidden from normal searches.
- Archiving is safer than deleting business records.

### Add active field

```
active = fields.Boolean(default=True)
```

### Archive records

```
records.action_archive()  
records.action_unarchive()
```

## 6.6 Read the Current User with env.user

- env.user is the current Odoo user.
- Use it for defaults and simple checks.
- Respect access rules when reading user data.

### Default salesperson

```
user_id = fields.Many2one('res.users', default=lambda self:
```

### Check groups

```
if self.env.user.has_group('sales_team.group_sale_manager'):  
    return True
```

## 6.7 Use sudo with Caution

- sudo bypasses normal access checks.
- Use it only when the business case is clear.
- Avoid exposing sudo data to the wrong user.

### Controlled sudo read

```
company = self.env['res.company'].sudo().browse(company_id)
```

### Prefer normal access

```
partners = self.env['res.partner'].search(domain)
```

# 7 Section 7

Create Related Record with [0, 0, vals] to Introduce CSV Access Rights

## 7.1 Create Related Record with [0, 0, vals]

- [0, 0, vals] creates a new related record.
- Use it in One2many or Many2many updates.
- vals contains the new record values.

### Create order line

```
order.write({'order_line': [(0, 0, {'name': 'Service', 'pr
```

### Create tag

```
record.write({'tag_ids': [(0, 0, {'name': 'New Tag'})]})
```

## 7.2 Update Related Record with [1, id, vals]

- [1, id, vals] updates an existing related record.
- The id is the related record id.
- Use vals for fields to change.

### Update line price

```
order.write({'order_line': [(1, line_id, {'price_unit': 12})]})
```

### Update tag name

```
record.write({'tag_ids': [(1, tag_id, {'name': 'Updated'})]})
```

## 7.3 Delete Related Record with [2, id]

- [2, id] removes and deletes the related record.
- Use it carefully with business data.
- It is different from only unlinking a relation.

### Delete order line

```
order.write({'order_line': [(2, line_id)]})
```

### Delete related tag

```
record.write({'tag_ids': [(2, tag_id)]})
```

## 7.4 Link Existing Record with [4, id]

- [4, id] links an existing record.
- It does not create or delete the linked record.
- It is common for Many2many fields.

### Add existing tag

```
record.write({'tag_ids': [(4, tag_id)]})
```

### Add existing follower group

```
record.write({'allowed_group_ids': [(4, group_id)]})
```



## 7.5 Replace Links with [6, 0, ids]

- [6, 0, ids] replaces all current links.
- ids is the full final list.
- Use it when syncing a complete selection.

### Replace tags

```
record.write({'tag_ids': [(6, 0, [1, 2, 3])])})
```

### Use ids from records

```
record.write({'user_ids': [(6, 0, users.ids)])})
```

## 7.6 Create an XML Data Record

- XML data creates or updates records during module load. [Simple XML record](#)
- The id becomes an external identifier.
- Keep data files listed in the manifest.

```
<record id="estate_tag_garden" model="estate.property_tag">  
  <field name="name">Garden</field>  
</record>
```

### Manifest data entry

```
'data': [  
    'views/estate_property_views.xml',  
    'data/estate_tags.xml',  
]
```

## 7.7 Introduce CSV Access Rights

- `ir.model.access.csv` grants model permissions.
- Access rights cover read, write, create, and delete.
- Every new model usually needs an access line.

### CSV header

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_
```

### Basic access row

```
access_estate_property_user,estate.property.user,model_est
```

NEXT STEP

# Try the examples on your machine

Then open the next module deck

---

Merit Advisory

04 · Odoo Core

